App Dev Project Report

1. Student Details

Name: Saurav Kumar
Roll Number: 24f3001092
Email: 24f3001092@ds.study.iitm.ac.in

About Me: I am a student pursuing an online degree with a strong interest in full-stack web development. This project gave me hands-on experience in designing and implementing a real-world multi-role web application using Flask, Python, SQLAlchemy, server-rendered templates, and role-based workflows.

2. Project Details

Project Title: Placement Portal

Problem Statement:

To design and build a web-based placement management system that connects students, companies, and an administrator in a single platform. The system should streamline campus recruitment activities such as registration, company approval, placement drive creation, student applications, profile management, and application status tracking.

Approach:

Built using Flask with Blueprint-based route separation for authentication, admin, company, student, and API functionality. Uses SQLAlchemy models for CRUD operations, Flask session management for role-based access control, and Flask-Bcrypt for secure password hashing. A default administrator account is auto-created on first run (admin@placement.com / admin123). UI implemented using Jinja2, Bootstrap 5, Bootstrap Icons, and custom CSS.

3. AI/LLM Declaration

The application does not implement any AI or LLM functionality. No AI-specific libraries (OpenAI, LangChain, Hugging Face, TensorFlow, PyTorch, Anthropic, Cohere, etc.) are imported or listed in dependencies. All major features — authentication, role management, admin approvals, placement drive management, student applications, resume handling, and JSON API responses — are implemented using Flask, SQLAlchemy, Flask-Bcrypt, Jinja2, Bootstrap, and standard Python utilities.

4. Technologies and Frameworks Used

Technology / Library	Source File(s)	Purpose
Flask	app.py, router.py, api.py	Core web framework — routing, sessions, Blueprint registration, request handling, redirects, flash messages, JSON responses
Flask-SQLAlchemy	models.py, app.py	ORM layer for defining models, relationships, queries, and database initialization
Flask-Bcrypt	models.py	Password hashing and verification via set_password() and check_password()
SQLite	app.py, placement_portal.db	File-based relational database for admins, companies, students, drives, and applications
Jinja2	templates/, router.py	Server-side HTML rendering for all role-based dashboards, forms, and listings
Bootstrap 5	templates/base.html	Responsive layout — cards, forms, tables, alerts, navbar, and modal UI components
Bootstrap Icons	templates/base.html	Iconography used across dashboards and navigation
Custom CSS	static/css/style.css	Application-specific styling for cards, navbar, drive cards, forms, and status display
Vanilla JavaScript	static/js/*.js	Frontend helpers for validation, chart rendering hooks, and notification behavior
Werkzeug	router.py	Safe file naming for uploaded resumes via secure_filename
Flask Session	router.py	Maintains logged-in user state with user_id, role, and name in session storage

5. Database Schema

Defined in models.py using SQLAlchemy ORM. Relationships: Company → PlacementDrive (one-to-many), Student → Application (one-to-many), PlacementDrive → Application (one-to-many). Cascade all, delete-orphan removes dependent records when the parent entity is deleted.

Table 1: admin

Field	Type	Constraints / Description
id	Integer	Primary key
username	String(50)	Unique, not null
email	String(120)	Unique, not null
password_hash	String(128)	Not null
created_at	DateTime	Default: datetime.utcnow

Table 2: company

Field	Type	Constraints / Description
id	Integer	Primary key
company_name	String(120)	Not null
email	String(120)	Unique, not null
hr_contact / website	String(20/200)	Nullable
industry / description	String(100) / Text	Nullable
approval_status	String(20)	Default: Pending (Pending, Approved, Rejected)
is_blacklisted	Boolean	Default: False — blocks access even if previously approved

Table 3: student

Field	Type	Constraints / Description
id	Integer	Primary key
name / email	String(100/120)	Not null; email unique
phone / department	String(20/100)	Nullable
cgpa / skills	Float / Text	Nullable
resume_filename	String(200)	Nullable — stored as email_originalfilename
is_blacklisted / is_active	Boolean	Default: False / True — blacklisting also deactivates account

Table 4: placement_drive

Field	Type	Constraints / Description
id	Integer	Primary key
company_id	Integer	Foreign key -> company.id, not null
job_title / job_description	String(150) / Text	Not null
eligibility_criteria / required_skills	Text / String(300)	Nullable
salary_range / location	String(100)	Nullable
application_deadline	Date	Not null
status	String(20)	Default: Pending (Pending, Approved, Closed, Rejected)

Table 5: application

Field	Type	Constraints / Description
id	Integer	Primary key
student_id / drive_id	Integer	FK -> student.id / FK -> placement_drive.id, not null
application_date	DateTime	Default: datetime.utcnow
status	String(20)	Default: Applied (Applied, Shortlisted, Selected, Rejected)
cover_note	Text	Nullable — UNIQUE(student_id, drive_id) prevents duplicate applications

6. API Resource Endpoints

Dedicated API Blueprint in api.py, registered under /api. All API routes are read-only GET endpoints returning JSON responses via jsonify().

Endpoint	Method	Data Returned
/api/students	GET	id, name, email, department, year_of_study for all students
/api/companies	GET	id, company_name, email, industry, approval_status for all companies
/api/drives	GET	id, company_id, job_title, status, application_deadline for all drives
/api/applications	GET	id, student_id, drive_id, status for all applications

Note: No separate api.yaml or openapi.yaml definition file is present in the project folder.

7. Architecture and Features

Project Structure

File / Folder	Description
app.py	Main Flask entry point; configures secret key, SQLite URI, upload folder (5 MB limit), registers Blueprints, creates tables, seeds default admin
router.py	Central routing module — Blueprint definitions, access-control decorators, auth/admin/company/student route handlers
models.py	SQLAlchemy models for Admin, Company, Student, PlacementDrive, Application with password helper methods
api.py	JSON API Blueprint exposing read-only resource endpoints
templates/	Jinja2 templates organized into auth/, admin/, company/, student/ directories
static/css/ & static/js/	Custom CSS for portal UI; JS helpers for validation, charts, notifications
static/uploads/resumes/	Uploaded student resumes stored for viewing and download
instance/placement_portal.db	SQLite database file generated by the application

Core Features

Module	Features
Authentication & Access Control	Multi-role login (Admin/Company/Student). Custom decorators: admin_required, company_required, student_required. Login blocked for pending/rejected/blacklisted companies and deactivated/blacklisted students. Logout clears session.
Admin Module	Dashboard with counts of students, companies, drives, and applications. Pending approvals visibility. Company management: approve, reject, blacklist/unblacklist, edit, delete. Student management: search, blacklist, edit, delete. Drive review: approve, reject, delete. Global applications view.
Company Module	Self-registration with HR contact, website, industry, description. Dashboard with drive and applicant counts. Create/edit/close/delete placement drives (edits reset status to Pending). View all applicants per drive. Update application status: Shortlisted, Selected, Rejected. Company profile page.
Student Module	Self-registration with academic details, skills, and optional resume upload. Browse only approved, non-expired drives. Search by job title or required skills. Apply with optional cover note. Duplicate application prevention at route and DB level. Track application history and status. Update profile and replace resume.
Frontend & UI	Responsive Bootstrap 5 layout — cards, tables, alerts, forms, navbar, modals. Role-aware navigation in base.html. Resume viewing for students and companies. Flash messages used throughout for action feedback.

8. Video

Project Demonstration Video Link: Add your Google Drive or YouTube project demo link here before final submission.

Summary: The Placement Portal is a multi-user campus recruitment system implemented with Flask, SQLAlchemy, SQLite, and Jinja2. It provides a complete workflow for company onboarding, admin approval, placement drive publishing, student applications, and application status management, all within a structured role-based web interface.